Compressed Sparse Arrays
27-Way Trees
Polymorphic Hashing
Cuckoo Hashing
Exact and Approximate (Bloom) Dictionaries
Binary Sparse Arrays
Cons, Car and Cdr

11. ADTs II & Data Structures

11.1 Compressed Sparse Arrays

Sometimes it is known in advance that you need an array-type data structure, but that it will be populated very sparsely i.e. very few of the locations will be used. This is extrememly memory inefficient; using one large array for this would be wasteful, since most of the memory would be redundant. Here we investigate one of the many ways of creating an alternative, where we trade-off speed for memory: Compact Sparse Arrays (CSA) which aim to be memory efficient for sparsely occupied arrays.

The main part of the CSA is a 'block' - a way of storing the values (and corresponding indices) of a section of the array compactly. A block contains:

- A 64-bit mask (represented using the type `uint64_t`) showing which of the cells of the arrays are in use.
- a (`realloc()`'d) integer array storing the values, ordered by their index.
- an index offset (a multiple of 64) showing what part of the overall array that block represents e.g. the block that contains indices 0...63 would have an offset of 0, block that contains indices 64...127 would have an offset of 64, and so on.

The CSA structure consists of a `realloc()`'d array of blocks (ordered by their `offset`), and a counter, n, storing the total number of blocks in use.

Exercise 11.1.1 Write an ADT to implement the `csa.h` given. Write the source files `mydefs.h` and `csa.c` such that the driver files `driver.c` and `fibmemo.c` work correctly. Ensure that this all works with my `Makefile` which **must be** used **unaltered**. The basic operations are :

```
csa* csa_init();
bool csa_set(csa* c, int idx, int val);
bool csa_get(csa* c, int idx, int* n);
void csa_tostring(csa* c, char* s);
void csa_free(csa** l);
```

The function `csa_set()` allows an integer to be written to a particular index. If it's the first index for a particular block, this will trigger the creation of the memory for this block.

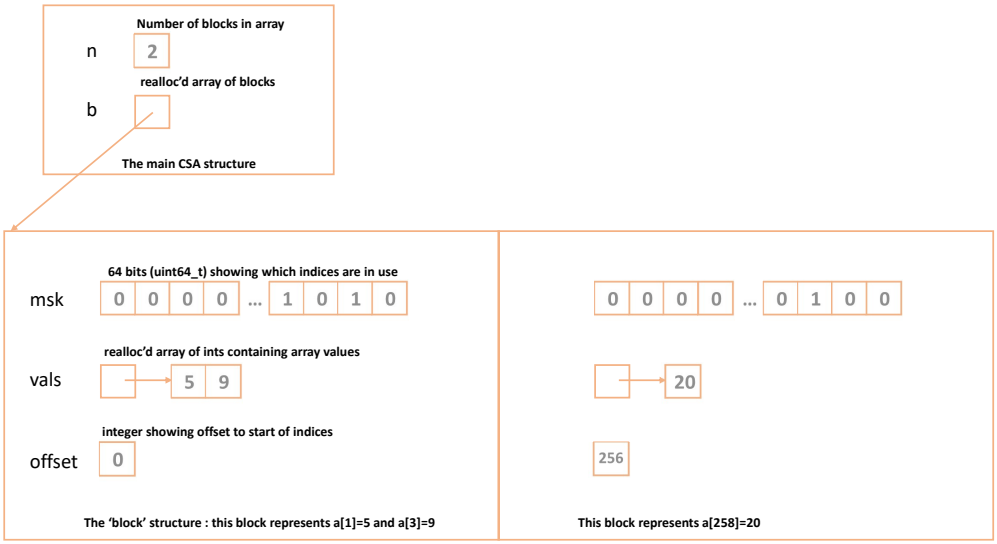


Figure 11.1: The Compressed Sparse Array. A dynamically allocated array of 'blocks' stores indices/values for the array, 64 values at a time.

`csa_get()` returns a Boolean showing whether `idx` is in use or not. If it is, the integer value is copied into n.
`csa_tostring()` allows a character based version of the CSA to be produced, showing each block in turn.

- You will use `realloc()` for the CSA - blocks are created only as and when required. No linked lists are used for this assignment.
- Even if you don't get all the functions to work correctly, make sure the code still compiles by writing 'dummy' functions as placeholders, even if some of the assertions fail.
- This assignment is brand new. If minor typos etc. are uncovered as you complete the assignment we may update this documentation and/or the online files provided. Please check regularly to make sure you are using the most recent version.

This is worth 80% **of the marks.**

Two extensions are availale, each worth 10%.

```
void csa_foreach(void (*func)(int* p, int* acc), csa* c, int* ac);
```

Allows the user to pass a function, which is in turn passed a pointer to all the values in the array. This would allow the user to, for instance, sum up the values, double them all etc. The `driver.c` file shows some examples of this, and `isfactorial.c` uses this to find a range of factorials.

```
bool csa_delete(csa* c, int idx);
```

Array elements can be deleted, and memory freed up if blocks are no longer used. If all the elements are deleted, you don't really need `csa_free()` since all the blocks have been freed, and simply calling `free()` on the csa structure should be enough. This allows the file `sieve.c` to be used to compute a range of prime numbers.

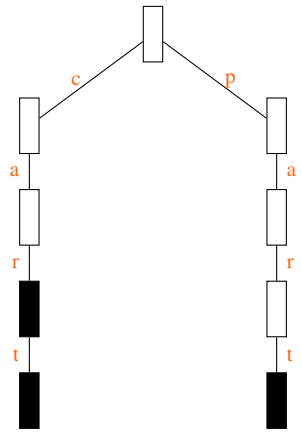
■

11.2 27-Way Trees

The Dictionary Abstract Data Type (ADT) allows words to be stored in such that they may be efficiently checked later, to ensure that words have been spelled correctly etc. There are many ways of implementing the Dictionary ADT (binary trees, hash tables and so on), but here we look at a tree structure which has a dynamic collection of nodes.
The 27-Way Tree has 27 downward links, each corresponding to one of the letters $a \ldots z$ and one to the apostrophe character '. This has many similarities with *tries*:

www `https://en.wikipedia.org/wiki/Trie`

Any word can be stored in the tree. You start at the top node, and, given the first character follow the relevant downward pointer. So, if the first character is 'a' you follow the first pointer down from the first node to another. If the next letter is 's' then you follow the 19^{th} downwards pointer from that node to the following one. Each pointer corresponds to a letter (or the apostrophe), and each level of the tree corresponds to the length of a word.



In the above diagram, the words 'car', 'cart' and 'part' have been stored in such a 27-Way Tree. Since 'car' is a substring of 'cart' we have to find a way of making it clear where valid word ending are. In the diagram, black-filled nodes show such *terminal* nodes. The string 'ca' is not a valid word since the block immediately afterwards has not been marked as such. The word 'par' is not valid either, since it has never been added to the dictionary (but could, in principle, be added later).

Exercise 11.2.1 Write an ADT to implement the `t27.h` interface given. Write the source file `t27.c` such that the driver file `driver.c` works correctly. Ensure that this all works with the `Makefile` which, as with `t27.h` **must be** used **unaltered**. The standard operations are :

```
dict* dict_init(void);
bool dict_addword(dict* p, const char* str);
```